

Paso 5 - Decidir | Solucionario

Las seis soluciones del paso 5. Varias muestran **más de una ejecución**, porque un `if` solo se prueba de verdad con los casos del borde.

Úsalo después de intentarlo. Cada solución viene con la salida real del programa: si la tuya coincide, está bien aunque el código sea distinto. Los `.java` de esta carpeta son los mismos que se muestran abajo, listos para ejecutar: `cd soluciones/paso05` y `javac *.java`.

B5.1 - Par o impar

Tres casos: el cero primero, porque si preguntas `% 2` antes, el cero cae en **par** y nunca llegas a tu mensaje.

▼ Ver la solución (B5_1_ParImpar.java)

B5_1_ParImpar.java

```
import java.util.Scanner;

public class B5_1_ParImpar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Numero: ");
        int numero = sc.nextInt();

        if (numero == 0) {
            System.out.println("El cero no cuenta");
        } else if (numero % 2 == 0) {
            System.out.println("Es par");
        } else {
            System.out.println("Es impar");
        }

        sc.close();
    }
}
```

Salida:

```
--- ejecucion 1
Numero: 0
El cero no cuenta

--- ejecucion 2
Numero: 8
Es par

--- ejecucion 3
Numero: -3
Es impar
```

Lo que importa: el orden. El caso especial (`== 0`) va **primero**. Si lo pones al final, nunca se ejecuta, porque el cero ya se fue por la rama de los pares. Esta es la regla general de los `else if`: **de lo más específico a lo más general**.

Fíjate en el `-3`: `-3 % 2` da `-1`, que no es 0, así que cae en impar y queda bien. Pero si hubieras escrito `numero % 2 == 1` en vez de `== 0`, el `-3` habría dado **par**. Es un bug real y muy común.

B5.2 - El tramo de edad

Cuatro tramos, con la validación **primero**.

▼ Ver la solución (B5_2_TramoEdad.java)

B5_2_TramoEdad.java

```
import java.util.Scanner;

public class B5_2_TramoEdad {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Edad: ");
        int edad = sc.nextInt();

        if (edad < 0 || edad > 120) {
            System.out.println("Esa edad no existe");
        } else if (edad < 18) {
            System.out.println("Menor de edad");
        } else if (edad < 65) {
            System.out.println("Adulto");
        } else {
            System.out.println("Adulto mayor");
        }

        sc.close();
    }
}
```

Salida:

```
--- ejecucion 1
Edad: -5
Esa edad no existe

--- ejecucion 2
Edad: 15
Menor de edad

--- ejecucion 3
Edad: 30
Adulto

--- ejecucion 4
Edad: 80
Adulto mayor

--- ejecucion 5
Edad: 130
Esa edad no existe
```

Lo que importa. Fíjate en que las condiciones **no repiten** el límite de abajo: la segunda dice solo `edad < 18`, no `edad >= 0 && edad < 18`. No hace falta, porque si llegó hasta ahí es que la primera no se cumplió. Cada `else if` ya tiene descartado todo lo anterior, y aprovecharlo hace el código mucho más corto.

Y la prueba del enunciado: si mueves el tramo `18 a 64` al principio, el `-5` responde `Adulto`... y el `130` también. Por eso la validación va primero.

B5.3 - La clave

Dos comparaciones de texto, y la lección del paso 5: con `==` **nunca** funciona.

▼ Ver la solución (B5_3_Clave.java)

B5_3_Clave.java

```
import java.util.Scanner;

public class B5_3_Clave {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Usuario: ");
        String usuario = sc.nextLine();

        System.out.print("Clave: ");
        String clave = sc.nextLine();

        // Con == esto SIEMPRE dice que la clave esta mala:
        // if (clave == "duoc2026") { ... }

        if (usuario.equalsIgnoreCase("alumno") && clave.equals("duoc2026")) {
            System.out.println("Bienvenido, " + usuario);
        } else {
            System.out.println("Usuario o clave incorrectos");
        }

        sc.close();
    }
}
```

Salida:

```
--- ejecucion 1
Usuario: ALUMNO
Clave: duoc2026
Bienvenido, ALUMNO

--- ejecucion 2
Usuario: alumno
Clave: DUOC2026
Usuario o clave incorrectos

--- ejecucion 3
Usuario: otro
Clave: duoc2026
Usuario o clave incorrectos
```

Por qué `equalsIgnoreCase` para el usuario y `equals` para la clave. El nombre de usuario es un identificador: da lo mismo cómo lo escriban, `ALUMNO` y `alumno` son la misma persona. La clave es un **secreto**: si aceptas `DUOC2026`, estás reduciendo a la mitad las combinaciones que tendría que probar alguien para adivinarla. Por eso la segunda ejecución tiene que fallar, aunque las letras sean las mismas.

Y el `&&`: las dos condiciones tienen que cumplirse. Java además es **perezoso**: si el usuario ya está mal, ni siquiera mira la clave.

B5.4 - La calculadora con switch

Un `switch` con cuatro `case`, su `default`, y un `if` adentro del caso 4 para la división por cero.

▼ Ver la solución (B5_4_Switch.java)

B5_4_Switch.java

```
import java.util.Scanner;

public class B5_4_Switch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Primer numero: ");
        double a = sc.nextDouble();

        System.out.print("Segundo numero: ");
        double b = sc.nextDouble();

        System.out.println("1) Sumar  2) Restar  3) Multiplicar  4) Dividir");
        System.out.print("Operacion: ");
        int operacion = sc.nextInt();

        switch (operacion) {
            case 1:
                System.out.println(a + " + " + b + " = " + (a + b));
                break;
            case 2:
                System.out.println(a + " - " + b + " = " + (a - b));
                break;
            case 3:
                System.out.println(a + " * " + b + " = " + (a * b));
                break;
            case 4:
                if (b == 0) {
                    System.out.println("No se puede dividir por cero");
                } else {
                    System.out.println(a + " / " + b + " = " + (a / b));
                }
                break;
            default:
                System.out.println("Esa operacion no existe");
        }

        sc.close();
    }
}
```

Salida:

```
--- ejecucion 1
Primer numero: 8
Segundo numero: 0
1) Sumar  2) Restar  3) Multiplicar  4) Dividir
Operacion: 4
No se puede dividir por cero

--- ejecucion 2
Primer numero: 8
Segundo numero: 2
1) Sumar  2) Restar  3) Multiplicar  4) Dividir
Operacion: 4
8.0 / 2.0 = 4.0

--- ejecucion 3
Primer numero: 8
Segundo numero: 2
1) Sumar  2) Restar  3) Multiplicar  4) Dividir
Operacion: 9
Esa operacion no existe
```

Lo que importa. El `if (b == 0)` va **dentro** del `case 4`: es el único lugar donde la división puede pasar. Validar antes, arriba del `switch`, obligaría a rechazar el 0 también para sumar, que no tiene nada de malo.

Una sorpresa para probar: si usas `int` en vez de `double`, `8 / 0` lanza `ArithmeticException: / by zero` y el programa muere. Pero con `double`, `8.0 / 0` **no** revienta: da `Infinity`. Java trata distinto la división entera y la decimal, y esa es una de las 12 formas de botar el programa de la demo 10.

B5.5 - El mayor de tres

Las dos formas en el mismo programa, para compararlas.

▼ Ver la solución (B5_5_MayorDeTres.java)

B5_5_MayorDeTres.java

```
import java.util.Scanner;

public class B5_5_MayorDeTres {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Primer numero: ");
        int a = sc.nextInt();
        System.out.print("Segundo numero: ");
        int b = sc.nextInt();
        System.out.print("Tercer numero: ");
        int c = sc.nextInt();

        // Forma 1: con if
        int mayor;
        if (a >= b && a >= c) {
            mayor = a;
        } else if (b >= c) {
            mayor = b;
        } else {
            mayor = c;
        }
        System.out.println("Con if:      " + mayor);

        // Forma 2: con Math.max
        System.out.println("Con Math.max: " + Math.max(Math.max(a, b), c));

        sc.close();
    }
}
```

Salida:

```
--- ejecucion 1
Primer numero: 3
Segundo numero: 9
Tercer numero: 5
Con if:          9
Con Math.max:    9

--- ejecucion 2
Primer numero: 7
Segundo numero: 7
Tercer numero: 2
Con if:          7
Con Math.max:    7
```

Las respuestas. La de `Math.max` se lee mejor y **no se puede equivocar**; la de `if` se entiende paso a paso pero tiene tres oportunidades de escribir mal un `>=`. Con diez números, la de `if` es inmanejable (habría que comparar todos contra todos) y la de `Math.max` se convierte en una cadena ilegible.

La solución de verdad para diez números es otra: guardarlos en un **arreglo** y recorrerlo con un `for` quedándote con el mayor, que es el B8.1. Un problema que con `if` no escala, con un ciclo cuesta cuatro líneas.

Ojo con los `>=` de la segunda ejecución: con `7, 7, 2` los dos primeros empatan, y por eso las comparaciones llevan `=`. Con `>` a secas, `a >= b && a >= c` fallaría y el empate se iría por otra rama (aquí daría lo mismo, pero en un caso con textos o índices, no).

B5.6 - Desafío: la nota en palabras

Cinco tramos con decimales. El orden vuelve a ser lo único difícil.

▼ Ver la solución (B5_6_NotaEnPalabras.java)

B5_6_NotaEnPalabras.java

```
import java.util.Scanner;

public class B5_6_NotaEnPalabras {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Nota (1.0 a 7.0): ");
        double nota = sc.nextDouble();

        if (nota < 1.0 || nota > 7.0) {
            System.out.println("Esa nota no existe");
        } else if (nota >= 6.5) {
            System.out.println("Excelente");
        } else if (nota >= 5.5) {
            System.out.println("Muy buena");
        } else if (nota >= 4.0) {
            System.out.println("Suficiente");
        } else {
            System.out.println("Insuficiente");
        }

        sc.close();
    }
}
```

Salida:

```
--- ejecucion 1
Nota (1.0 a 7.0): 6.8
Excelente

--- ejecucion 2
Nota (1.0 a 7.0): 5.5
Muy buena

--- ejecucion 3
Nota (1.0 a 7.0): 4.0
Suficiente

--- ejecucion 4
Nota (1.0 a 7.0): 3.9
Insuficiente

--- ejecucion 5
Nota (1.0 a 7.0): 7.5
Esa nota no existe
```

La respuesta del enunciado: con `switch` no se puede. El `switch` compara con valores **exactos** (`case 1:`, `case 2:`), y aquí las condiciones son **rangos**: entre 5.5 y 6.4 hay infinitos decimales y no puedes escribir un `case` para cada uno. Rangos → `if`. Valores exactos → `switch`.

Fíjate en que los tramos van **de mayor a menor** y cada uno solo pregunta por su límite de abajo. Si los ordenas al revés, todo cae en el primer `if` que se cumple y todas las notas salen `Suficiente`.

Volver a la [hoja del paso 5](#) o al [paso 5 de la guía](#).